INDIAN INSTITUTE OF INFORMATION TECHNOLOGY NAGPUR
Department of Computer Science & Engineering
**CSL 205: Design and Analysis of Algorithms**

**Date**: 19th Feb, 2026 (Thu)

Exam: Mid Term Exam
Duration: 1 Hr: 30 Min | Semester – IV (CSE/ CSH/ CSD/ CSA) | Max. Marks: 30

***Important Instructions:***

- This is a closed book, closed notes examination.
- This question paper comprises total 5 questions.
- All the questions are compulsory.

Q.1

a) How many times is the comparison $i >= n$ performed in the following program? (2M) CO1

```
int i = 300, n = 150;
main() {
  while (i >= n){
    i = i - 2;
    n = n + 1;
  }
}
```

Ans: 52

(2M) CO1

b) An AI surveillance system monitors an $n \times n$ security grid. The grid is recursively divided into four equal zones and each zone is analyzed for suspicious activity. After recursive processing, a cross-zone consistency check is performed along internal boundaries, taking time proportional to $n$.
If suspicious activity density is high, the system recursively analyzes a central hotspot region of size $n/4 \times n/4$. Assuming constant time for the smallest unit:
Construct the recurrence relation, $T(n)$. Also, state whether it is homogeneous or non-homogeneous.

Ans: (2M) CO1

**Given**

- Input: an $n \times n$ grid
- The grid is divided into **four equal zones**
  → each subproblem has size $\frac{n}{2} \times \frac{n}{2}$
- Each zone is processed **recursively**
- After recursion, a **cross-zone consistency check** is done
  → takes time proportional to $n$
- If suspicious activity is high, the system **recursively analyzes one central hotspot** of size $\frac{n}{4} \times \frac{n}{4}$
- Base case: constant time for the smallest unit



Scanned with OKEN Scanner

**Step 1: Identify recursive calls**

1. From four equal zones:

$$4T(n/2)$$

2. From the central hotspot (additional recursive call):

$$T(n/4)$$

---

**Step 2: Identify non-recursive work**

- Boundary / consistency check costs:

$$\Theta(n)$$

---

**Step 3: Construct the recurrence relation**

Combining all parts:

$$\boxed{T(n) = 4T(n/2) + T(n/4) + \Theta(n)}$$

with

$$T(1) = \Theta(1)$$

**The recurrence is non- Homogeneous**

c) Take the following list of functions and arrange then in ascending order of growth rate. That is if function *g(n)* immediately follows function *f(n)* in your list, then it should be the case that *f(n)* is *O(g(n))*.

$$f_1(n) = 10^n, f_2(n) = n^{1/3}, f_3(n) = n^n, f_4(n) = log_2^n, f_5(n) = 2^{\sqrt{log_2^n}}$$

Ans:

f4 <= f5 <= f2 <= f1 <= f3

Q.2 a) What is the value returned by the function following function? Express your answer as a function of n. Give the exact running time of the function. (3M) CO1

```
r = 0;
for i = 1 to n do
  for j = 1 to i do
    for k = j to i+j do
      for l = 1 to j+i-k do
          r = r +1;
```

Ans: (3M) CO1

$$\sum_{i=1}^{n}\sum_{j=1}^{i}\sum_{k=j}^{i+j}(j+i-k).$$



Scanned with OKEN Scanner

**Simplify inner two loops**

Let

$k = j + t$, where $t = 0 \ldots i$.

Then

$$j + i - k = j + i - (j + t) = i - t.$$

So for fixed $i, j$:

$$\sum_{k=j}^{i+j} (j + i - k) = \sum_{t=0}^{i} (i - t) = i + (i-1) + \cdots + 0 = \frac{i(i+1)}{2}.$$

This does not depend on j.

**Sum over $j$**

$$\sum_{j=1}^{i} \frac{i(i+1)}{2} = i \cdot \frac{i(i+1)}{2} = \frac{i^2(i+1)}{2}.$$

---

**Sum over $i$**

$$r = \sum_{i=1}^{n} \frac{i^2(i+1)}{2} = \frac{1}{2} \sum_{i=1}^{n} (i^3 + i^2).$$

$$\sum i^3 = \frac{n^2(n+1)^2}{4}, \qquad \sum i^2 = \frac{n(n+1)(2n+1)}{6}.$$

$$r = \frac{1}{2} \left[ \frac{n^2(n+1)^2}{4} + \frac{n(n+1)(2n+1)}{6} \right].$$

Factor and simplify:

$$\boxed{r = \frac{n(n+1)(3n^2 + 7n + 2)}{24}}$$

b) For the pair of functions, *f(n), g(n)* determine which of the following holds

$f(n) = \Theta(g(n))$ *or*

$f(n) = O(g(n))$ *but* $f(n) \neq \Theta(g(n))$ *or*

$f(n) = \Omega(g(n))$ *but* $f(n) \neq \Theta(g(n))$

*i)* $f(n) = n^2 / \log n$, $g(n) = n (\log n)^2$ and *ii)* $f(n) = (\log n)^{\log n}$, $g(n) = n / \log n$

*Ans:*

*i)* $f(n) = \Omega(g(n))$ *but* $f(n) \neq \Theta(g(n))$

*ii)* $f(n) = \Omega(g(n))$ *but* $f(n) \neq \Theta(g(n))$

| | | |
|---|---|---|
| Q.3 | a) Solve the recurrence relation using substitution method<br><br>$T(n) = 2T(n/2) + n(\lg n)^2$, $n > 1$, and $T(1) = 1$.<br><br>Ans: | (2 ½M) CO1 |



Scanned with OKEN Scanner

$$T(n) = 2T(n/2) + n(\lg n)^2$$

$$= 2\left[2T(n/4) + \frac{n}{2}(\lg(n/2))^2\right] + n(\lg n)^2$$

$$= 4T(n/4) + 2 \cdot \frac{n}{2}(\lg(n/2))^2 + n(\lg n)^2$$

$$= 4T(n/4) + n(\lg(n/2))^2 + n(\lg n)^2$$

(2 ½M) CO1

(3M) CO1

After $i$ expansions:

$$T(n) = 2^i T(n/2^i) + n \sum_{j=0}^{i-1} (\lg(n/2^j))^2.$$

---

## 2. Simplify logarithm term

$$\lg(n/2^j) = \lg n - j.$$

$$T(n) = 2^i T(n/2^i) + n \sum_{j=0}^{i-1} (\lg n - j)^2.$$

---

## 3. Stop when base case reached

Stop when $n/2^i = 1 \Rightarrow i = \lg n$.

Then

$$2^i T(n/2^i) = 2^{\lg n} T(1) = n.$$

Let $m = \lg n$. The sum becomes

$$\sum_{j=0}^{m-1} (m-j)^2 = \sum_{k=1}^{m} k^2 \downarrow \frac{m(m+1)(2m+1)}{6}.$$



Scanned with OKEN Scanner

$$T(n) = n + n \cdot \frac{m(m+1)(2m+1)}{6}, \quad m = \lg n.$$

$$\boxed{T(n) = n\left[1 + \frac{(\lg n)(\lg n + 1)(2\lg n + 1)}{6}\right]}$$

---

### 5. Asymptotic growth

Highest-order term:

$$(\lg n)(\lg n + 1)(2\lg n + 1) \sim 2(\lg n)^3.$$

$$\boxed{T(n) = \Theta\big(n(\lg n)^3\big)}.$$

b) Solve the recurrence relation using Master Method

$$T(n) = 4T(n/4) + n\log n,\ n > 1, \text{ and } T(1) = 1.$$

Ans:

- $a = 4$
- $b = 4$
- $f(n) = n \log n$

---

### Step 2: Compute critical exponent

$$n^{\log_b a} = n^{\log_4 4} = n^1 = n.$$

---

### Step 3: Compare $f(n)$ with $n^{\log_b a}$

$$f(n) = n \log n = n^{\log_b a} \log^1 n.$$

This matches **Master Theorem Case 2**:

$$f(n) = \Theta\big(n^{\log_b a} \log^k n\big).$$

Here $k = 1$.

---

### Step 4: Apply Case 2 formula

For Case 2:

$$T(n) = \Theta\big(n^{\log_b a} \log^{k+1} n\big).$$

So

$$T(n) = \Theta(n \log^2 n).$$

c) Solve the recurrence relation

$$T(n) = 7T(n-1) - 10T(n-2) + (2n+5)3^n$$



Scanned with OKEN Scanner

Q.4

Ans:

$(x^2 - 7x + 10)(x - 3)^{1+1} = 0$

$(x - 2)(x - 5)(x - 3)^2 = 0$

$T(n) = c1\ n^2 + c2\ 5^n + c3\ 5^n + c4\ n3^n$

$$\boxed{T(n) = \frac{100}{3}5^n - \frac{4}{3}2^n - 9(n+3)3^n}$$

or factored:

$$\boxed{T(n) = \frac{1}{3}\left[100 \cdot 5^n - 4 \cdot 2^n - 27(n+3)3^n\right].}$$

(2M) CO1

a) Consider an array A with n distinct integers where $n > 2$. In an array, for some index $p$ between $0 < p < n - 1$, the values in the array elements increase up to index $p$ from index 0 and then decrease the remainder of the way until position $n - 1$. Consider the following algorithm **find_max** to return the index of the maximum element of array A:

```
function find_max(A):
    low = 0
    high = length(A)-1
    while True:
        mid = (low + high)//2
        if (Condition1):
            low = mid
        elif (Condition2):
            high = mid
        else:
            return mid
```

(3M) CO2

What should be the Condition1 and Condition2 so that the algorithm correctly returns the required output?

Ans:

```
Condition1 -> (A[mid-1] < A[mid]) and (A[mid] < A[mid+1])
Condition2 -> (A[mid-1] > A[mid]) and (A[mid] > A[mid+1])
```

b) Consider a data structure where we perform n operations. The cost of the $i$th operation is $i$ if $i$ is a power of 2; otherwise, the cost of operation $i$ is 1. Use the accounting method.

Ans:

## Step 1: Understand expensive operations

Operations costing more than 1 occur when $i$ is a power of 2:

$$1, 2, 4, 8, 16, \ldots, 2^k \leq n.$$

Number of such operations:

$$\lfloor \log_2 n \rfloor + 1.$$



Scanned with OKEN Scanner

## Step 2: Total actual cost

Split into two parts:

### (A) Cost of non-power-of-2 operations

There are about $n - \log n$ of them, each costs 1:

$$\approx n - \log n.$$

### (B) Cost of power-of-2 operations

Sum of powers of 2:

$$1 + 2 + 4 + 8 + \cdots + 2^k$$

This is a geometric series:

$$= 2^{k+1} - 1.$$

Since $2^k \leq n < 2^{k+1}$,

$$2^{k+1} < 2n.$$

So power-of-2 cost < $2n$.

### Total cost

$$T(n) < (n - \log n) + 2n = 3n - \log n.$$

$$T(n) = O(n).$$

## Step 3: Accounting Method

We assign an **amortized cost** per operation so that total amortized ≥ total real cost.

Charge each operation **3 units**.

---

### Check balance

Total charged:

$$3n.$$

Actual cost:

$$< 3n.$$

### Amortized cost per operation

$$\boxed{O(1)} \quad (\text{specifically} \leq 3).$$

### Total cost of n operations

$$\boxed{O(n)}.$$



Scanned with OKEN Scanner

| | | |
|---|---|---|
| Q.5 | a) An algorithm solves a problem by repeatedly reorganizing the input around a chosen element, after which the resulting parts are handled independently using the same logic. The process continues until trivial cases are reached. Identify the underlying algorithmic strategy and explain why this method improves efficiency as the input size grows.<br>Ans:<br><br>**Underlying algorithmic strategy:**<br>The algorithm follows the **Divide and Conquer** strategy.<br><br>**Explanation of efficiency:** The algorithm improves efficiency because each step **breaks the original problem into smaller independent subproblems,** reducing the input size handled at later stages. Solving smaller problems recursively requires less work than processing the entire input at once, leading to **better scalability and lower overall time complexity** as the input size grows.<br><br>*This reduction in problem size at each step is the key reason for improved performance.* | (1M) CO1<br><br>(2M) CO1 , CO3<br><br>(2M) CO1 , CO3 |
| | b) Consider the recurrence $f(n) = f(n-1) * f(n-1)$, $f(1) = 2$. Show using induction that $f(n)$ is $\Omega(2^n)$.<br>Ans:<br>*I.H. For all* $j < n$, $f(j) \geq c2^n$<br>*I.S.* $f(n) = f(n\text{-}1)*f(n\text{-}1)$. *By the I.H.,* $f(n) \geq c^2 2^{n-1} * 2^{n-1} \geq c2^n$ *provided that* $c \geq 1$. *Thus, if* $c = 1$ *and* $n_0 = 1$, *we can say that* $f(n) \geq c2^n$ *for all* $n \geq n_0$ *which proves that* $f(n) = \Omega(2^n)$. | |
| | c) Prove that $o(g(n)) \cap w(g(n))$ is the empty set? The symbol $o$ is the little/small-Oh, and $w$ is the small-Omega.<br>Ans:<br>**Little-o**<br>$$f(n) \in o(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0.$$<br>**Little-ω**<br>$$f(n) \in \omega(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty.$$ | |



Scanned with OKEN Scanner

## Proof (by contradiction)

Assume there exists a function $f(n)$ such that

$$f(n) \in o(g(n)) \cap \omega(g(n)).$$

Then both must hold:

1.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$$

2.

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty.$$

But a single limit **cannot simultaneously equal 0 and ∞**.

This is a contradiction.

---

## Conclusion

No function can belong to both sets.

$$\boxed{o(g(n)) \cap \omega(g(n)) = \varnothing}$$



Scanned with OKEN Scanner